OFFICIAL REFERENCE BOOK • EDITION V1.6.0

The Kaz Programming Language Book 🦅

The Definitive Unified Guide to the Language, 14 Native Stdlib Modules, Sovereign x86-64 AOT Compilers (ELF64 & PE32+), and the 7-Phase .kzm AI Engine

VERSION	**Kaz 1.6.0 (Sovereign AOT & Native AI Edition)**
CREATOR & AUTHOR	Armando Soares (@armandosds)
EXECUTION ENGINES	Stack Bytecode VM • Cranelift JIT • Sovereign x86-64 AOT (ELF64 & PE32+)
AI ARCHITECTURE	Zero-RAM mmap .kzm v2 • KZQ4/KZQ8 Block Quantization • AVX2 + FMA3 SIMD
OFFICIAL PORTAL	https://kazlang.xyz • https://github.com/kazlang/kaz

© 2026 Armando Soares & Kazlang Project

The Kaz Programming Language Book 🦅

The Official Comprehensive Guide & Technical Reference — Version 1.6.0

Author: Armando Soares (`armandosds`)

Official Website: https://kazlang.xyz

Official Repository: https://github.com/kazlang/kaz

Edition: Kaz 1.6.0 (Sovereign ELF64 / PE32+ Native Edition)

📖 Table of Contents

1. **Chapter 1: Introduction & Architectural Philosophy**

- 1.1 What is Kaz?
- 1.2 The Four Pillars of Kaz Engineering
- 1.3 The Three Execution Engines (Sovereign AOT, Cranelift JIT, Stack VM)

1. **Chapter 2: Getting Started & Ecosystem Setup**

- 2.1 Installation (Linux, Windows, macOS)
- 2.2 Verifying Your Installation
- 2.3 Your First Program: `Hello, World!` and `runoff()`
- 2.4 IDE Support (VS Code & Lumina IDE) and TermiKAZ Companion

1. **Chapter 3: Lexical Structure, Primitive Types & Variables**

- 3.1 Comments & Statements
- 3.2 Strict Static Type System
- 3.3 The 5 Canonical Rules of Primitive Types
- 3.4 Exact Financial Fixed-Point Arithmetic (`decimal`)
- 3.5 Variables, Immutability (`const`), and Lexical Scope

1. **Chapter 4: Operators, Expressions & Control Flow**

- 4.1 Arithmetic, Relational, and Short-Circuit Logical Operators
- 4.2 The Ternary Conditional Operator (`? :`)
- 4.3 Conditional Branching (`if`, `else if`, `else`)
- 4.4 Modern Iteration (`for..in`, `.indices`, C-style `for`, `while`, `break`, `continue`)

1. **Chapter 5: Functions, Procedures & Exception Handling**

- 5.1 Procedures vs. Typed Return Functions
- 5.2 The Canonical `Main()` Entry Point
- 5.3 Deep Recursion & Stack Frame Optimization
- 5.4 Zero-Overhead Exception Handling (`try / catch` & Stack Unwinding)

1. **Chapter 6: Composite & Algebraic Data Types**

- 6.1 Typed Structures (`struct`)
- 6.2 Algebraic Tagged Unions (`enum` with Payloads)
- 6.3 Exhaustive Pattern Matching (`match`)
- 6.4 Contiguous Dynamic Arrays (`array[T]` and `array[any]`)

1. **Chapter 7: Modular Programming & Project Scaffolding**

- 7.1 Multi-File Architecture with `import`
- 7.2 Anti-Cycle Canonical Module Resolution
- 7.3 Instant Project Scaffolding (`kaz new` / `kaz create`)

1. [**Chapter 8: The Batteries-Included Standard Library (`Stdlib`)**](#chapter-8-the-batteries-included-standard-library-stdlib)

- 8.1 Console I/O, CLI Arguments & Assertions (`io`)
- 8.2 Type Conversions & Introspection (`convert`)
- 8.3 Hardware-Accelerated Mathematics (`math.*`)
- 8.4 UTF-8 String Manipulation (`str`)
- 8.5 Array & Collection Methods (`arr`)
- 8.6 Synchronous File System (`fs.*`)
- 8.7 Cryptography, Hashing & Encoding (`crypto.*`)
- 8.8 Regular Expressions (`regex.*`)
- 8.9 Networking, TCP Sockets & HTTP REST (`net.*`)
- 8.10 Embedded Relational SQLite Database (`db.*`)
- 8.11 Native JSON Serialization & Parsing (`json.*`)
- 8.12 High-Resolution Time & System (`time`)
- 8.13 Event Hooks & Sandboxed Modding (`hook`)
- 8.14 Raw Memory Buffers (`buf_*`)

1. **Chapter 9: CLI Tooling, Canonical Formatter & Unit Testing**

- 9.1 Running, Checking, and Building Code
- 9.2 AST-Validated Canonical Code Formatter (`kaz fmt`)
- 9.3 Built-In Unit Testing Framework (`kaz test`)
- 9.4 Diagnostics: Stack VM Tracer (`kaz trace`), Disassembler (`kaz debug`), and SQL Console (`kaz db-cli`)
- 9.5 Interactive Kaz Terminal Shell (`kaz shell`)

1. **Chapter 10: Sovereign Native Compilation (Linux ELF64 & Windows PE32+)**

- 10.1 Zero External Toolchain Philosophy (No NASM, GCC, Clang, LD, or MSVC)
- 10.2 Sovereign Linux x86-64 ELF64 Backend
- 10.3 Sovereign Windows x86-64 PE32+ (`.exe`) Backend
- 10.4 Self-Hosting & Fixed-Point Mathematical Proof (Stage 3 == Stage 4)

1. **Chapter 11: Memory Architecture, Atomic ARC & Security Guarantees**

- 11.1 Sandbox Isolation Rule: Bytecode VM vs. Native JIT/AOT
- 11.2 Atomic Reference Counting (`AtomicUsize` 16-Byte Header)

- 11.3 Dynamic Heap Arena via Direct Syscall (`sys_brk`)
- 11.4 Preventing Remote DoS in Long-Running Daemons (`heap_checkpoint` & `heap_rewind`)
- 11.5 Active Heap Ceiling & Anti-OOM Protection (`heap_set_limit`)

1. [**Chapter 12: The Sovereign AI Engine (`.kzm`, 7-Phase AOT & KZQ Quantization)**](#chapter-12-the-sovereign-ai-engine-kzm-7-phase-aot--kzq-quantization)

- 12.1 Architecture of the `.kzm` Binary Format (64-Byte Aligned Zero-Copy `mmap`)
- 12.2 The 7-Phase AI Model AOT Compiler (`.safetensors` $\rightarrow$ `.kzm`)
- 12.3 KZQ Hardware-Aligned Quantization Formats
- 12.4 Native AI Tensor & Tokenizer Stdlib Reference
- 12.5 Official Benchmarks: OLMo-2-1B Ground-Truth Evaluation vs. `llama.cpp` (GGUF)

Chapter 1: Introduction & Architectural Philosophy

1.1 What is Kaz?

Kaz is an independent, sovereign, statically-typed compiled systems programming language created from scratch by **Armando Soares**. It is engineered to unite the expressive clarity of modern high-level languages with the uncompromising bare-metal speed of native systems programming.

Unlike conventional ecosystems that depend on sprawling external C/C++ toolchains, external linkers, gigabytes of runtime dependencies, or third-party package managers for basic tasks, Kaz is **100% self-contained**:

- It includes its own **Self-Hosted Compiler** (`compiler/*.kaz`), proven deterministic via bit-for-bit Fixed-Point validation.
- It emits standalone native **Linux x86-64 ELF64** and **Windows x86-64 PE32+ (`.exe`)** binaries directly from memory without invoking NASM, GCC, Clang, GNU LD, or MSVC.
- It features a **Cranelift JIT Compiler**, a **Stack Bytecode Virtual Machine** (implemented both in Rust and in 100% pure self-hosted Kaz), an embedded relational database engine (**SQLite**), native HTTP/TCP networking, cryptography, and a **Sovereign SIMD AI Tensor Runtime (`.kzm`)** capable of running multi-billion-parameter Large Language Models on pure CPU silicon.

1.2 The Four Pillars of Kaz Engineering

1. **Technological Sovereignty**: Zero reliance on `libc`, external assemblers, or Python/C++ wrappers. A compiled Kaz binary communicates directly with the operating system kernel.
2. **Strict Type Safety**: Compile-time and runtime type enforcement across primitives, algebraic tagged unions (`enum`), structures (`struct`), and homogeneous collections (`array[T]`).
3. **Deterministic Memory without GC Pauses**: Memory is managed via Atomic Reference Counting (ARC), segmented L1-cache Slab Free-Lists, and 64-byte aligned Bump Arenas with $O(1)$ scope checkpoints (`heap_checkpoint` / `heap_rewind`).
4. **True Batteries Included**: Database persistence (`db.*`), JSON (`json.*`), cryptography (`crypto.*`), regular expressions (`regex.*`), file system (`fs.*`), unit testing (`kaz test`), canonical formatting (`kaz fmt`), and LLM tensor inference (`model.*`) are built directly into the single `kaz` executable.

1.3 The Three Execution Engines

Kaz provides three complementary execution modes tailored for every stage of software engineering:

Execution Mode	CLI Command	Startup Time	Performance Profile	Primary Use Case
1. Sovereign Native AOT	`kaz build app.kaz -o app`	**< 1 ms**	**Bare-Metal x86-64** (~10 KB static ELF64 / PE32+)	Production binaries, microservices, CLI tools, OS utilities
2. Cranelift JIT Engine	`kaz jit app.kaz`	**~2 ms**	**~1.9x of Native Rust (`-O3`)** (~39x faster than Python)	Compute-intensive scripts, numerical loops, real-time execution
3. Stack Bytecode VM	`kaz run app.kaz`	**< 3 ms**	**27x faster than AST interpreters**	Rapid development, debugging, tracing, sandboxed plugins/mods

---

Chapter 2: Getting Started & Ecosystem Setup

2.1 Installation (Linux, Windows, macOS)

Linux / macOS (Automated One-Liner)

```
curl -fsSL https://kazlang.xyz/install.sh | sh
```

Debian / Ubuntu (`.deb` Package)

```
sudo dpkg -i kaz_1.6.0_amd64.deb
```

Windows (PowerShell Automated Installer or Winget)

```
irm https://kazlang.xyz/install.ps1 | iex
# Or via Winget:
winget install Armando.KazLang
```

2.2 Verifying Your Installation

Open any terminal and run:

```
kaz --version
# Output: Kaz 1.6.0
```

2.3 Your First Program: `Hello, World!` and `runoff()`

In Kaz, the primordial output primitive is `runoff()`. Unlike bloated formatting macros in other languages that pull in megabytes of locale and synchronization machinery, `runoff()` streams directly to standard output (`stdout`).

Create a file named `hello.kaz`:

```
// hello.kaz - First program in Kaz 🦅
function Main() {
    string message = "Hello, World! Welcome to Kaz v1.6.0 🦅";
    runoff(message);
}
```

Run it immediately in the Bytecode VM, with the JIT compiler, or compile it to a standalone native binary:

```
# 1. Run via Stack Bytecode VM:
kaz run hello.kaz

# 2. Run with Cranelift JIT native acceleration:
kaz jit hello.kaz

# 3. Compile to a standalone native executable:
kaz build hello.kaz -o hello
./hello
```

2.4 IDE Support (VS Code & Lumina IDE) and TermiKAZ Companion

- **Syntax Highlighting & Snippets (`.vsix`): Install the official extension for Visual Studio Code or** Lumina IDE**:

```
  code --install-extension Kaz-Language.kaz-language-1.6.0.vsix
```

- **TermiKAZ Companion**: The official terminal emulator engineered for Kaz developers, featuring a built-in POSIX/Linux shell, smart `[TAB]` autocompletion for Kaz subcommands, and one-click toolbar actions for `JIT`, `Fmt`, `Test`, and `Audit`.

Chapter 3: Lexical Structure, Primitive Types & Variables

3.1 Comments & Statements

Every statement in Kaz terminates with a semicolon (`;`). Code blocks are enclosed in curly braces (`{ }`).

```
// Single-line comment

/*
   Multi-line
   block comment
*/

runoff("Kaz syntax is clean, predictable, and unambiguous.");
```

3.2 Strict Static Type System

Type Keyword	Description	Literal Example	Default Value
`int` / `i64`	64-bit signed integer (two's complement wrapping)	`int count = 42;`	`0`
`u8`, `i8`, `u16`, `i16`, `u32`, `i32`	Explicit fixed-width integer types	`u8 byte_val = 255;`	`0`
`float` / `double`	64-bit IEEE-754 double-precision floating point	`float pi = 3.14159265;`	`0.0`
`f32`	32-bit IEEE-754 single-precision floating point	`f32 weight = 0.25;`	`0.0`
`decimal`	Exact 64-bit fixed-point decimal (10^4 scale, 4 decimal places)	`decimal price = 19.99m;`	`0.0m`
`string`	UTF-8 encoded text string	`string name = "Kaz 🦅";`	`""`
`char`	32-bit Unicode scalar character (`u32`)	`char grade = 'A';`	`'\0'`
`bool` / `bol`	Logical boolean	`bool active = true;`	`false`
`array[T]`	Homogeneous dynamically-sized typed array	`array[int] ids = [1, 2, 3];`	`[]`
`array[any]`	Heterogeneous dynamic array	`array[any] mix = [1, "two", true];`	`[]`
`struct`	User-defined composite record	`User u = User { ... };`	Null fields
`void`	Explicit absence of a return value	`function log(): void { ... }`	`void`

3.3 The 5 Canonical Rules of Primitive Types

1. **Floating-Point Precision**: `float` and `double` are strictly 64-bit IEEE-754 (`f64`). `f32` is 32-bit single-precision.
2. **Text & Unicode**: `char` is a 32-bit Unicode scalar value (`u32`). `string` is a contiguous UTF-8 byte sequence; lengths and offsets measure raw UTF-8 bytes.
3. **Null Safety in Structs**: Uninitialized struct pointers default to `null` (`0`). Attempting to read or mutate fields on a null struct triggers a deterministic runtime panic rather than undefined memory corruption.
4. **Integer Arithmetic**: Integer operations use 64-bit two's complement wrapping (`wrapping_add` , `wrapping_sub` , `wrapping_mul`). Division or modulo by zero triggers a deterministic panic (`kaz_panic_divzero`).
5. **Exact Decimal Arithmetic**: The `decimal` type uses an implicit scale factor of $10,000$ (10^4), eliminating binary floating-point representation errors (`0.1m + 0.2m == 0.3m` is strictly `true`).

3.4 Exact Financial Fixed-Point Arithmetic (`decimal`)

Binary floating-point (`float`) cannot represent base-10 fractions like `0.1` or `0.01` exactly, leading to rounding bugs in financial software. Kaz solves this natively in hardware with the `decimal` primitive type (suffixed with `m` or

`M`):

```
decimal unit_price = 12.34m;
decimal tax_rate = 0.05m;
decimal total = unit_price * (1.00m + tax_rate);
runoff("Exact financial total:", total);
```

In the x86-64 AOT compiler, `decimal` multiplication and division use the 128-bit hardware register pair `rdx:rax` (`imul` followed by `idiv 10000`), preventing intermediate 64-bit overflow while executing at bare-metal silicon speed.

3.5 Variables, Immutability (`const`), and Lexical Scope

```
// Mutable variables
int port = 8080;
string host = "127.0.0.1";

// Immutable constants (protected at compile-time and runtime)
const float MAX_TIMEOUT = 30.0;
string const API_VERSION = "v1.6.0";

// Lexical block scoping
if (port == 8080) {
    int worker_threads = 4;
    runoff("Starting server on " + host + ":" + port + " with " + worker_threads + " threads.");
}
// 'worker_threads' is automatically reclaimed outside the block
```

Chapter 4: Operators, Expressions & Control Flow

4.1 Arithmetic, Relational, and Short-Circuit Logical Operators

- **Arithmetic**: `+`, `-`, `*`, `/`, `%`
- **Bitwise & Shifts**: `<<`, `>>`, `&`, `|`, `^`
- **Compound Assignment**: `+=`, `-=`, `*=`, `/=`, `%=`
- **Increment / Decrement**: `x++`, `x--`
- **Relational Comparison**: `==`, `!=`, `<`, `<=`, `>`, `>=`
- **Logical (Short-Circuiting)**: `&&` (AND), `||` (OR), `!` (NOT)
- **String Concatenation**: `+` automatically converts primitive operands to string when concatenated with a `string`.

4.2 The Ternary Conditional Operator (`? :`)

```
int latency_ms = 12;
string tier = (latency_ms < 20) ? "Ultra-Fast" : (latency_ms < 100) ? "Normal" : "Slow";
runoff("Network Tier: " + tier);
```

4.3 Conditional Branching (`if`, `else if`, `else`)

```
int score = 92;

if (score >= 90) {
    runoff("Grade: A (Excellence)");
} else if (score >= 75) {
    runoff("Grade: B (Approved)");
} else {
    runoff("Grade: C (Review Required)");
}
```

4.4 Modern Iteration (`for..in`, `.indices`, C-style `for`, `while`, `break`, `continue`)

1. Iterating Over Elements (`for..in`)

```
array[string] targets = ["ELF64", "PE32+", "Cranelift-JIT", "KZM-SIMD"];

for (t in targets) {
    runoff("Active Backend: " + t);
}
```

2. Iterating Over Valid Indices (`.indices`)

```
for (idx in targets.indices) {
    runoff("Target [" + idx + "] = " + targets[idx]);
}
```

3. Classic C-Style `for` Loop

```
int sum = 0;
for (int i = 1; i <= 100; i++) {
    sum += i;
}
runoff("Gauss Sum (1..100): " + sum); // 5050
```

4. `while` Loop with `break` and `continue`

```
int step = 0;
while (true) {
    step++;
    if (step % 2 != 0) {
        continue; // Skip odd numbers
    }
    if (step > 6) {
        break; // Exit loop
    }
    runoff("Processed even step: " + step);
}
```

Chapter 5: Functions, Procedures & Exception Handling

5.1 Procedures vs. Typed Return Functions

Kaz distinguishes between **procedures** (which perform side effects and return no value) and **typed functions** (which declare an explicit `: ReturnType` after the parameter list):

```
// Procedure (no return value, or explicit : void)
function logBanner(string title) {
    runoff("========================================");
    runoff("  " + title);
    runoff("========================================");
}

// Typed Function (explicit : float return type)
function computeDiscount(float price, float rate): float {
    return price * (1.0 - rate);
}

logBanner("Billing Engine");
float final_price = computeDiscount(200.0, 0.15);
runoff("Final Price: $" + final_price);
```

5.2 The Canonical `Main()` Entry Point

When a file or project defines `function Main()` (or `function main()`), the Kaz compiler automatically designates it as the program entry point after initializing all imported modules and global constants:

```
function Main(): int {
    runoff("System initialized.");
    return 0; // Returned as the OS process exit code in sovereign ELF64/PE32+ binaries!
}
```

5.3 Deep Recursion & Stack Frame Optimization

```
function fibonacci(int n): int {
    if (n <= 1) {
        return n;
    }
    return fibonacci(n - 1) + fibonacci(n - 2);
}

runoff("Fibonacci(26) = " + fibonacci(26)); // Executes in 995 microseconds with Cranelift JIT!
```

5.4 Zero-Overhead Exception Handling (`try / catch` & Stack Unwinding)

Kaz provides structured exception handling via `try / catch` blocks. In the Stack Bytecode VM (`PushTry` / `PopTry`), this mechanism incurs **zero runtime overhead** along the happy path and performs automatic **Call Stack Unwinding** across arbitrarily deep function calls when a runtime error occurs (such as division by zero, missing file, failed SQL constraint, network timeout, or out-of-bounds array access):

```
function loadConfigFile(string path): string {
    return fs.read(path); // Throws runtime exception if file does not exist
}

function initializeSystem() {
    try {
        string raw = loadConfigFile("non_existent_config.json");
        runoff("Config loaded: " + raw);
    } catch (err) {
        runoff("[RECOVERED] Using default configuration. Reason: " + err);
    }
}

initializeSystem();
```

If you do not need the error message string, the parameter in `catch` can be omitted (`catch { ... }`).

Chapter 6: Composite & Algebraic Data Types

6.1 Typed Structures (`struct`)

Structures group named, statically-typed fields into a contiguous record:

```
struct ServerNode {
    int id;
    string hostname;
    int port;
    bool healthy;
}

ServerNode node = ServerNode {
    id: 1,
    hostname: "node-alpha.kazlang.xyz",
    port: 9000,
    healthy: true
};

runoff("Node " + node.hostname + " listening on port " + node.port);
node.port = 9090; // Direct field mutation
```

6.2 Algebraic Tagged Unions (`enum` with Payloads)

Kaz supports Rust-style algebraic data types (Tagged Unions), allowing each variant of an `enum` to carry typed payload data:

```
enum ConnectionState {
    Connected,
    Reconnecting(int),
    Failed(string),
}

ConnectionState s1 = ConnectionState::Connected;
ConnectionState s2 = ConnectionState::Reconnecting(3);
ConnectionState s3 = ConnectionState::Failed("TLS Handshake Timeout");
```

6.3 Exhaustive Pattern Matching (`match`)

The `match` statement safely destructures `enum` variants, extracts payload variables, or matches primitive literals with a wildcard fallback (`_`):

```
function describeState(ConnectionState st): string {
    string report = "";
    match (st) {
        ConnectionState::Connected => {
            report = "Link is online and healthy.";
        }
        ConnectionState::Reconnecting(attempt) => {
            report = "Reconnection attempt #" + attempt + " in progress...";
        }
        ConnectionState::Failed(reason) => {
            report = "Critical connection failure: " + reason;
        }
        _ => {
            report = "Unknown state.";
        }
    }
    return report;
}

runoff(describeState(s2));
runoff(describeState(s3));
```

6.4 Contiguous Dynamic Arrays (`array[T]` and `array[any]`)

> **Canonical Array Type Syntax**: In Kaz, array types are always declared using the canonical prefix generic syntax `array[T]` (for example, `array[int]`, `array[string]`, `array[ServerNode]`), never C-style `T[]`.

```
array[string] events = ["BOOT", "AUTH_OK"];
events.push("DB_CONNECTED");

runoff("Total events: " + events.len);      // 3
runoff("Joined: " + events.join(" -> "));   // "BOOT -> AUTH_OK -> DB_CONNECTED"
bool hasAuth = events.contains("AUTH_OK");  // true
string last = events.pop();                 // Removes and returns "DB_CONNECTED"
```

Chapter 7: Modular Programming & Project Scaffolding

7.1 Multi-File Architecture with `import`

Kaz is designed for large-scale software engineering across structured directories. Use `import "relative/path.kaz";` to bring structs, enums, constants, and functions from other modules into scope:

```
// File: src/models/user.kaz
struct User {
    int id;
    string name;
}

function createUser(int id, string name): User {
    return User { id: id, name: name };
}
```

```
// File: src/main.kaz
import "models/user.kaz";

function Main() {
    User u = createUser(101, "Ada Lovelace");
    runoff("Created user: " + u.name + " (#" + u.id + ")");
}
```

7.2 Anti-Cycle Canonical Module Resolution

1. **File-Relative Resolution**: When `src/main.kaz` imports `"models/user.kaz"`, Kaz resolves the path relative to the directory containing `src/main.kaz`, regardless of your current working directory in the terminal.
2. **Idempotent Anti-Cycle Cache**: The compiler maintains a normalized canonical path cache. If Module A imports Module B and Module B imports Module A (diamond or circular dependency), each file is parsed and compiled exactly once.
3. **Directory Execution**: You can execute an entire project folder directly:

```
   kaz my_project/
   # or inside the project folder:
   kaz .
```

7.3 Instant Project Scaffolding (`kaz new` / `kaz create`)

Generate a complete production project layout with a single command:

```
kaz new my_system
cd my_system
kaz run .
kaz test
```

Generated architecture:

```
my_system/
├── kaz.json             # Project configuration manifest
├── README.md            # Quickstart instructions
├── .gitignore           # Standard ignore rules (*.db, bin/, dist/)
├── data/
│   └── schema.sql       # Relational SQLite DDL schema
├── src/
│   ├── main.kaz         # Application entry point
│   ├── database.kaz     # SQLite connection & migration helpers
│   └── models/
│       └── usuario.kaz  # Typed domain models
└── tests/
    └── main_test.kaz    # Automated native unit tests
```

Chapter 8: The Batteries-Included Standard Library (`Stdlib`)

All 14 Standard Library modules are compiled directly into the `kaz` runtime binary—requiring zero external packages or C libraries.

8.1 Console I/O, CLI Arguments & Assertions (`io`)

Function	Parameters	Return	Description
`runoff(...)`	`...values: any`	`void`	Canonical output instruction: prints values separated by spaces followed by a newline.
`print(...)`	`...values: any`	`void`	Prints values to `stdout` without a trailing newline.
`input(prompt?)`	`prompt?: string`	`string`	Displays optional prompt and reads a line from `stdin`.
`input_int(prompt?)`	`prompt?: string`	`int`	Displays optional prompt and parses an integer from `stdin`.
`read_line()`	*(none)*	`string`	Reads a line of text from `stdin`.
`args_len()`	*(none)*	`int`	Returns the number of command-line arguments passed to the program.
`argv(index)`	`index: int`	`string`	Returns the command-line argument at `index` as a `string`.
`argv_int(index)`	`index: int`	`int`	Returns the command-line argument at `index` parsed as an `int`.
`assert(cond, msg?)`	`cond: bool, msg?: string`	`bool`	Validates `cond`. Panics with an assertion error if `false`.

8.2 Type Conversions & Introspection (`convert`)

Function	Parameters	Return	Description
`to_int(val)`	`val: any`	`int`	Converts string, float, or bool to 64-bit integer.
`to_float(val)`	`val: any`	`float`	Converts string, int, or bool to 64-bit float.
`to_string(val)`	`val: any`	`string`	Converts any value, array, or struct to its string representation.
`to_bool(val)`	`val: any`	`bool`	Converts number or string to boolean.
`type_of(val)`	`val: any`	`string`	Returns the runtime type name (`"int"`, `"string"`, `"array"`, or struct name).
`ord(ch)` / `chr(code)`	`string` / `int`	`int` / `string`	Converts between ASCII/UTF-8 character and integer byte value.
`f32_to_bits(f)` / `f32_from_bits(b)`	`float` / `int`	`int` / `float`	Raw IEEE-754 32-bit float bit reinterpretation.

8.3 Hardware-Accelerated Mathematics (`math.*`)

Callable via `math.<fn>()` or directly in global scope:

- `math.sqrt(x)` *(emits native hardware `sqrtsd` instruction in JIT/AOT)*
- `math.pow(base, exp)`, `math.abs(x)`, `math.random()`
- `math.sin(rad)`, `math.cos(rad)`, `math.tan(rad)`
- `math.floor(x)`, `math.ceil(x)`, `math.round(x)`
- `math.min(a, b)`, `math.max(a, b)`

8.4 UTF-8 String Manipulation (`str`)

Callable as methods on strings (`s.trim()`) or standalone functions (`str_trim(s)`):

- `s.len` / `str_len(s)`: Length in bytes.
- `s.upper()` / `str_upper(s)`, `s.lower()` / `str_lower(s)`
- `s.trim()` / `str_trim(s)`
- `s.split(delim)` / `str_split(s, delim)`: Returns `array[string]`.
- `s.replace(old, new)` / `str_replace(s, old, new)`
- `s.contains(sub)` / `str_contains(s, sub)`
- `s.starts_with(pre)` / `str_starts_with(s, pre)`, `s.ends_with(suf)` / `str_ends_with(s, suf)`
- `s.substring(start, end)` / `str_substring(s, start, end)`

8.5 Array & Collection Methods (`arr`)

- `arr.len` / `len(arr)`: Number of elements.
- `arr.indices`: Returns `array[int]` of valid indices `0 .. arr.len - 1`.
- `arr.push(item)`, `arr.pop()`, `arr.join(sep)`, `arr.contains(item)`, `arr.reverse()`.

8.6 Synchronous File System (`fs.*`)

- `fs.mkdir(path)`: Recursively creates directories (`mkdir -p`).
- `fs.read_dir(path)`: Returns `array[string]` of directory entries.
- `fs.read(path)` / `file_read(path)`: Reads entire file as UTF-8 string.
- `fs.write(path, content)` / `file_write(path, content)`: Writes string to file.
- `fs.append(path, content)` / `file_append(path, content)`: Appends string to file.
- `fs.exists(path)` / `file_exists(path)`: Returns `true` if file or directory exists.
- `fs.remove(path)` / `file_delete(path)`: Deletes file or directory.
- `fs.copy(src, dst)`: Copies file from `src` to `dst`.
- `fs.size(path)`: Returns file size in bytes.
- `file_read_bytes(path)` / `file_write_bytes(path, bytes)`: Reads or writes raw binary byte arrays (`array[int]`).

8.7 Cryptography, Hashing & Encoding (`crypto.*`)

- `crypto.sha256(text)`: Returns 64-character hexadecimal SHA-256 digest.
- `crypto.md5(text)`: Returns 32-character hexadecimal MD5 digest.
- `crypto.base64_encode(text)` / `crypto.base64_decode(b64_text)`: Standard Base64 encoding and decoding.

8.8 Regular Expressions (`regex.*`)

- `regex.is_match(pattern, text)`: Returns `bool`.
- `regex.find(pattern, text)`: Returns first matched substring (or `""`).
- `regex.replace(pattern, text, replacement)`: Replaces all matches of `pattern` in `text`.

8.9 Networking, TCP Sockets & HTTP REST (`net.*`)

- `net.ping(host, port, timeout_ms?)`: Measures TCP handshake latency in milliseconds (`-1` on error).
- `net.http_get(url)`: Performs HTTP GET request and returns response body.
- `net.http_post(url, body, content_type?)`: Performs HTTP POST request with payload.
- `net.tcp_send(host, port, message)`: Sends raw TCP payload and returns response string.

8.10 Embedded Relational SQLite Database (`db.*`)

```
int conn = db.open("data/production.db"); // Or ":memory:" for in-RAM database
db.execute(conn, "CREATE TABLE IF NOT EXISTS users (id INTEGER PRIMARY KEY, name TEXT, balance
REAL);");
db.execute(conn, "INSERT INTO users (name, balance) VALUES ('Armando', 2500.50);");

array[any] rows = db.query(conn, "SELECT id, name, balance FROM users;");
for (row in rows) {
    runoff("User #" + row.id + ": " + row.name + " ($" + row.balance + ")");
}
db.close(conn);
```

8.11 Native JSON Serialization & Parsing (`json.*`)

- `json.stringify(value, pretty?)`: Serializes any Kaz struct, array, or primitive into JSON.
- `json.parse(json_str)`: Parses a JSON string into navigable Kaz objects and arrays.

8.12 High-Resolution Time & System (`time`)

- `time_now_us()`: Monotonic high-resolution timestamp in microseconds (μs).
- `time_now_ms()`: Current timestamp in milliseconds (ms).
- `time_now_secs()`: Current UNIX timestamp in fractional seconds (`float`).
- `sleep_ms(ms)`: Suspends thread execution for `ms` milliseconds.

8.13 Event Hooks & Sandboxed Modding (`hook`)

- `hook.register(event_name, handler_fn_name)`
- `hook.has(event_name)`, `hook.list(event_name)`, `hook.clear(event_name)`

8.14 Raw Memory Buffers (`buf_*`)

Engineered for zero-overhead binary serialization, network packets, and AI tensor arenas with magic-tag (`0x4B415A42` `'KAZB'`) bounds checking:

- `buf_alloc(size_bytes)` / `buf_free(buf_handle)` / `buf_size(buf_handle)`
- `buf_load_u8(h, off)`, `buf_store_u8(h, off, val)`
- `buf_load_u16_le(h, off)`, `buf_store_u16_le(h, off, val)`
- `buf_load_u32_le(h, off)`, `buf_store_u32_le(h, off, val)`
- `buf_load_i64_le(h, off)`, `buf_store_i64_le(h, off, val)`
- `buf_load_f32_le(h, off)`, `buf_store_f32_le(h, off, val)`
- `buf_copy(dst_h, dst_off, src_h, src_off, len)`

Chapter 9: CLI Tooling, Canonical Formatter & Unit Testing

9.1 Running, Checking, and Building Code

Command	Description
`kaz <path>` / `kaz run <path>`	Compiles and executes `.kaz` file or project folder in the Stack Bytecode VM.
`kaz jit <file.kaz>`	Compiles and executes `.kaz` file using the native **Cranelift JIT** engine.
`kaz build <file.kaz> -o <bin>`	Compiles `.kaz` file to a standalone native executable (`--native`, `--bundle`, `--emit-obj`).
`kaz check <path>`	Validates syntax, imports, and types across files or folders without executing.
`kaz fmt [path] [--check]`	Formats `.kaz` files canonically (K&R 1TBS, 4 spaces, AST-verified).
`kaz test [path]`	Runs native `test "..." { ... }` blocks and reports pass/fail timing.
`kaz trace <file.kaz>`	Traces Stack VM execution instruction-by-instruction with live operand stack inspection.
`kaz debug <file.kaz>`	Disassembles compiled bytecode chunks and constant pools without executing.
`kaz db-cli <file.db>`	Opens interactive SQL console for inspecting `.db` / SQLite databases.
`kaz shell` / `kaz`	Launches the interactive Kaz Terminal Shell with built-in POSIX commands.
`kaz repl`	Launches lightweight expression REPL.

Passing Arguments to Scripts: Use `--` to separate `kaz` flags from script arguments:
```bash
kaz run my_script.kaz -- arg1 arg2
```

9.2 AST-Validated Canonical Code Formatter (`kaz fmt`)

`kaz fmt` enforces a single, uniform style across the entire Kaz ecosystem:

- 4 spaces per indentation level, K&R / 1TBS brace placement, uniform spacing around binary and ternary operators, and preservation of single-line and block comments.
- **AST Integrity Guarantee**: Before writing formatted code back to disk, `kaz fmt` parses the formatted output and compares its Abstract Syntax Tree (AST) against the original file's AST. If any syntactic divergence is detected, the write is aborted, guaranteeing `kaz fmt` can never corrupt user logic.

9.3 Built-In Unit Testing Framework (`kaz test`)

Write `test` blocks directly inside your source files or in dedicated `tests/*_test.kaz` files. During normal execution (`kaz run`), `test` blocks are ignored; during `kaz test`, they are automatically discovered and executed:

```
function multiply(int a, int b): int {
    return a * b;
}

test "multiplication identity and zero property" {
    assert(multiply(7, 6) == 42, "7 * 6 must equal 42");
    assert(multiply(999, 0) == 0, "Multiplication by zero must return 0");
}
```

Chapter 10: Sovereign Native Compilation (Linux ELF64 & Windows PE32+)

10.1 Zero External Toolchain Philosophy

Version **1.6.0** achieves complete compiler emancipation. A true sovereign compiler should not secretly shell out to `gcc`, `clang`, `nasm`, `ld`, or `link.exe` behind the scenes. Kaz implements its own binary object emitters and machine code encoders in pure Kaz:

- **`compiler/backend/elf64/`**: Sovereign Linux x86-64 ELF64 backend.
- **`compiler/backend/pe64/`**: Sovereign Windows x86-64 PE32+ (`.exe`) backend.
- **`tools/aot_soberano.kaz`**: Unified dual-target CLI compiler & `.kzm` model packager.

10.2 Sovereign Linux x86-64 ELF64 Backend

```
Kaz Source (.kaz) ⟶ Pratt Parser (AST) ⟶ Scalar IR Lowering ⟶ Native x86-64 Encoder
⟶ Relocation Fixups (.text, .rodata, .data) ⟶ Sovereign _start Stub ⟶ ELF64 Binary
```

- **Direct ELF64 Construction**: Emits the 64-byte ELF64 header (`0x7F 'E' 'L' 'F'`) and 56-byte `PT_LOAD` program headers aligned to 4096-byte virtual memory pages (`0x400000`).
- **Sovereign `_start` Bootstrap**: Captures `argc` from `[rsp]` and `argv` from `rsp + 8`, invokes `Main()`, moves the return value into `rdi`, and invokes Linux syscall `60` (`sys_exit`) directly via `0f 05` (`syscall`).
- **Result**: Self-contained executables starting at **217 bytes to ~10 KB**, requiring zero `libc` or dynamic loader on the target Linux machine.

10.3 Sovereign Windows x86-64 PE32+ (`.exe`) Backend

Implemented in `compiler/backend/pe64/pe64_writer.kaz` and `tools/aot_soberano.kaz`, Kaz generates native 64-bit Windows executables directly from any OS without MSVC or MinGW:

- **DOS Header (`MZ`) & `e_lfanew` Pointer**: 64-byte header pointing directly to `PE\0\0`.
- **COFF File Header (`0x8664`) & PE32+ Optional Header (`0x020B`)**: Configured for `IMAGE_SUBSYSTEM_WINDOWS_CUI` with 512-byte file alignment and 4096-byte section alignment at standard 64-bit image base `0x140000000`.
- **Handcrafted Import Address Table (IAT) & Import Directory (`.rdata`)**: Automatically synthesizes the Import Lookup Table and Hint/Name thunks linking `kernel32.dll` (`GetStdHandle`, `WriteFile`, `ExitProcess`) via RIP-relative indirect calls (`FF 15 disp32`).
- **Result**: A 1,536-byte `.exe` binary that runs natively on Windows 10/11 x64 with zero external linker.

10.4 Self-Hosting & Fixed-Point Mathematical Proof (Stage 3 == Stage 4)

The self-hosted compiler (`compiler/*.kaz`) completed formal multi-stage bootstrap verification:

1. **Stage 1 $\rightarrow$ Stage 2**: Host compiler compiles `compiler/*.kaz` into native autonomous binary `kazc_stage2` (269 KB).
2. **Stage 2 $\rightarrow$ Stage 3**: `kazc_stage2` compiles `compiler/*.kaz` on its own, producing `kazc_stage3.asm` (2,309,919 bytes) in 39 seconds.
3. **Stage 3 $\rightarrow$ Stage 4 (Fixed Point)**: `kazc_stage3` recompiles `compiler/*.kaz` to produce `kazc_stage4.asm`.
4. **Bit-for-Bit Identity**: `kazc_stage3.asm` and `kazc_stage4.asm` are 100% identical byte-for-byte (`SHA-256: 95398a3ce2056ca043767774130fa07593da2f7b3b7ad5e0ad265679a29d7ae8`).

Furthermore, `compiler/vm.kaz` implements the complete 88-opcode Stack Bytecode VM in just 798 lines of pure Kaz.

Chapter 11: Memory Architecture, Atomic ARC & Security Guarantees

11.1 Sandbox Isolation Rule: Bytecode VM vs. Native JIT/AOT

> **Architectural Security Policy for Third-Party Scripts & Mods**:
> Untrusted third-party scripts, user plugins, and runtime mods must be executed inside the **Kaz Stack Bytecode VM (`kaz run` / `kaz vm`)**, where every memory access, array index, and stack frame is bounds-checked by Rust RAII guarantees. Native JIT (`kaz jit`) and AOT (`kaz build`) operate directly on hardware memory pointers for maximum throughput.

11.2 Atomic Reference Counting (`AtomicUsize` 16-Byte Header)

In native JIT execution, heap objects carry a standardized 16-byte prefix header:

- `Offset -16`: `ref_count` managed via thread-safe `AtomicUsize` (`AcqRel` / `Release` memory ordering).

- `Offset -8`: `type_id` (`u64` index into the `RwLock`-protected Dynamic Drop Registry for recursive field destruction).
- `Offset 0`: Start of struct/array payload.

combined with a thread-local **L1-Cache Slab Free-List** that recycles small struct allocations without hitting OS `malloc`.

11.3 Dynamic Heap Arena via Direct Syscall (`sys_brk`)

In Sovereign AOT binaries, Kaz eliminates bloated static `.bss` reservations. The `.bss` section is only 24 bytes (`kaz_heap_start`, `kaz_heap_current`, `kaz_heap_limit`), and `kaz_alloc` expands virtual memory on demand in **64 MB chunks** via direct Linux syscall `12` (`sys_brk`), strictly aligned to **64-byte cache-line boundaries** (`(size + 63) & -64`) for AVX2/AVX-512 SIMD compatibility.

11.4 Preventing Remote DoS in Long-Running Daemons (`heap_checkpoint` & `heap_rewind`)

In long-running network daemons (`net_listen`) or interactive terminals, allocating dynamic arrays inside an infinite request loop would cause monotonic heap growth. Kaz solves this with $O(1)$ transactional arena checkpoints:

```
while (true) {
    int cp = heap_checkpoint();      // Capture current arena pointer in O(1)
    int client = net_accept(server);
    handleClientRequest(client);     // Freely allocate temporary strings, structs, and arrays
    heap_rewind(cp);                 // Instantly reclaim 100% of request memory in O(1) with zero GC
pause!
}
```

11.5 Active Heap Ceiling & Anti-OOM Protection (`heap_set_limit`)

To protect Linux hosts where `vm.overcommit_memory = 0` allows processes to exhaust physical RAM and swap, the Kaz AOT runtime enforces an active heap ceiling (default 4 GB, configurable at runtime via `heap_set_limit(bytes)`), panicking cleanly with exit code `1` before triggering host OS thrashing.

Chapter 12: The Sovereign AI Engine (`.kzm`, 7-Phase AOT & KZQ Quantization)

12.1 Architecture of the `.kzm` Binary Format (64-Byte Aligned Zero-Copy `mmap`)

The ** `.kzm` (Kaz Model Format)** is a sovereign binary container designed for instant zero-copy memory mapping (`mmap`) with zero initial RAM overhead:

- **64-Byte Header**: Magic bytes `KZM1`, format version, architecture ID, layer count, hidden dimension, attention heads, vocabulary size, and tensor directory offset.
- **128-Byte Tensor Directory Entries**: UTF-8 tensor name (96 bytes), quantization type (`u32`), rank (`u32`), shape dimensions, 64-byte aligned `file_offset` (`u64`), and `data_size` (`u64`).

- **64-Byte Aligned Payload Sections**: Every tensor and AOT section begins on a strict 64-byte boundary (`offset % 64 == 0`), allowing AVX2 (`_mm256_load_ps`) and AVX-512 vector instructions to stream directly from disk-mapped pages.

12.2 The 7-Phase AI Model AOT Compiler (`.safetensors` $\rightarrow$ `.kzm`)

Located in `compiler/ai/` (and tested via `kaz test tests/ai/aot_compiler_test.kaz`), Kaz includes a complete 7-phase Ahead-of-Time compiler for neural networks written in pure Kaz:

1. **Phase 1 (`phase1_ingest.kaz`)**: Parses standard HuggingFace `.safetensors` headers and constructs the Directed Acyclic Graph (DAG).
2. **Phase 2 (`phase2_hir.kaz`)**: Lowers the DAG into High-Level IR (HIR) with strict tensor shape and type inference.
3. **Phase 3 (`phase3_graph_opt.kaz`)**: Hardware-independent graph optimizations:

- *Constant Folding* of static scale/norm parameters.
- *Operator Fusion* (`Conv2D + BatchNorm + Bias + ReLU`, `Linear + Bias + ReLU`, `RoPE + GQA Attention`, `SwiGLU MLP`).
- *Dead Code Elimination (DCE)* pruning unused probe/debug tensors.

1. **Phase 4 (`phase4_lir.kaz`)**: Lowers optimized HIR into Low-Level Tensor IR (TIR) with L1/L2 cache blocking (`tile_m`, `tile_k`) and 4x FMA loop unrolling schedules.
2. **Phase 5 (`phase5_target_opt.kaz`): Runs the** Liveness-Based Static Memory Planner**, reusing intermediate activation buffers to drastically reduce peak RAM, and generates the x86-64 AVX2 register plan.
3. **Phase 6 (`phase6_codegen.kaz`)**: Emits native x86-64 AVX2 machine code and quantizes weights into aligned blocks.
4. **Phase 7 (`phase7_kzm_export.kaz`): Packages** Section 1 (`__aot_blueprint__`)**,** Section 2 (`__aot_machine_code__`)**, and** Section 3 (64B-Aligned Weights)** into a validated `.kzm` artifact and executes it in a **Zero-Malloc Runtime Arena** (achieving **624x to 6,485x speedup** over scalar VM execution).

12.3 KZQ Hardware-Aligned Quantization Formats

Quantization ID	Name	Block Structure (per 32 weights)	Bits per Weight (bpw)	Description
`0`	** `FP32` **	128 bytes (IEEE-754 float32)	32.00 bpw	Uncompressed ground-truth baseline
`1`	** `BF16` **	64 bytes (Brain Float 16)	16.00 bpw	Half-precision dynamic range
`2`	** `KZQ-8` **	34 bytes (2B BF16 scale + 32B `int8`)	8.50 bpw	Near-lossless 8-bit quantization
`3`	** `KZQ-4B` **	18 bytes (2B BF16 scale + 16B packed 4-bit nibbles)	**4.50 bpw**	Balanced 4-bit uniform quantization
`4`	** `KZQ-4S` **	14 bytes (MSE-optimized hybrid 4-bit / 3-bit)	**3.68 bpw**	Ultra-compact smart hybrid quantization
`5`	** `KZQ-2X` **	12 bytes (Extreme 2-bit / 3-bit sub-byte packing)	~3.00 bpw	Extreme edge compression

12.4 Native AI Tensor & Tokenizer Stdlib Reference

Function	Description
`model_open(path)` / `kzm_open(path)`	Maps a `.kzm` or `.safetensors` model into memory and returns its handle.
`kzm_validate(path)`	Verifies magic header, directory bounds, and 64-byte alignment integrity.
`kzm_layers(h)`, `kzm_dim(h)`, `kzm_heads(h)`, `kzm_vocab(h)`	Inspects model architecture hyperparameters.
`kzm_tensor_names(path)`, `kzm_tensor_info(path, name)`	Lists and inspects individual tensor metadata, shape, and file offsets.
`kzm_tensor_ptr(h, name)`, `kzm_tensor_size(h, name)`, `kzm_tensor_qtype(h, name)`	Zero-copy pointer and descriptor access for high-speed layer execution.
`mat_vec_mul_auto(w_ptr, x_buf, out_buf, rows, cols, qtype)`	Multi-threaded AVX2 + FMA (`hsum_avx`) matrix-vector multiplication across any KZQ format.
`rms_norm_f32(out, x, w, n, eps)`	Vectorized Root Mean Square Layer Normalization.
`silu_mul_f32(out, gate, up, n)`	Fused SwiGLU (`SiLU(gate) * up`) activation kernel.
`rope_f32(x, pos, heads, head_dim, rot_dim, theta)`	Rotary Positional Embeddings (RoPE).
`tokenizer_load(json_path)`	Loads a HuggingFace `tokenizer.json` BPE vocabulary and regex splitter.
`tokenizer_encode(tok_h, text)` / `tokenizer_id_to_token(tok_h, id)`	Encodes text to token IDs (`100%` parity with `llama-tokenize`) and decodes IDs to strings.

12.5 Official Benchmarks: OLMo-2-1B Ground-Truth Evaluation vs. `llama.cpp` (GGUF)

Evaluated on **AllenAI `OLMo-2-0425-1B`** (1,484,965,888 parameters) across the 100-Question Ground-Truth Benchmark ($T=0.0$, frozen scorer `score.kaz` / `score.py` passing 19/19 self-tests) on an Intel Haswell 4C/8T CPU @ 2.60 GHz (AVX2/FMA):

1. **100% Engine Concordance (Kaz FP32 vs. GGUF F32)**:

- Input Token IDs: **100 / 100 identical** (`SHA-256: 3a57d7b7c59eac802c8284db3c5bb8d8e702f0b8e51d9e484beccf95e70ea780`).
- First-Line Generated Answers: **100 / 100 identical**.
- Ground-Truth Score: **53 / 100 (53.0%)** on both engines (Exact McNemar $p = 1.000$).

1. **Quantized Size & Accuracy**:

- **Kaz `KZQ-4B` (`olmo-2-1b-kzq4.kzm`): 797.0 MiB **(4.50 bpw) —** 43 / 100 (81.1% retention)**, statistically indistinguishable from `GGUF Q4_0` (850.9 MiB, $p = 0.688$) and matching `GGUF Q3_K_M` (43 / 100, $p = 1.000$) while being **95.2 MiB smaller** than `GGUF Q4_K_M`.

1. **Decode Throughput & SMT Stability**:

- **FP32 Decode**: `2.40 tok/s` (Kaz) vs. `2.41 tok/s` (`llama.cpp`), saturating `~14.3 GB/s` of the `15.5 GB/s` DDR3 hardware bus ceiling.
- **KZQ-4B Decode**: **9.09 – 9.93 tok/s** with POSIX barrier thread-pool scaling cleanly across 8 SMT threads without the hyperthreading collapse observed in `llama.cpp`.

© 2026 Armando Soares. Kaz Programming Language — Sovereign Native Systems Architecture.